What Problem Does the New Agent Service Solve?

InfraFlow — GenAI Multi-Agent Architecture Overview

The Problem

InfraFlow's original “AI Cloud Agent” was a single RAG bot that answered generic AWS Well-Architected questions from static documentation — it never looked at the user's own diagram. This change replaces it with a multi-agent system that reasons over the user's real infrastructure: their actual diagram graph, generated Terraform, validation errors, deployment plan, and (if connected) real AWS billing data. It answers architecture, Terraform-correctness, cost, and security questions grounded in that live state, with citations back to real node IDs / Terraform addresses instead of invented facts. This is the “GenAI AI agent” / “self-deploying architecture” initiative.

Main Components

services/agents/ (new Python / FastAPI service, port 8001)

The reasoning layer. It holds **no** AWS credentials, no Terraform binary, and no DB driver by design (enforced by tests/test_no_cloud_sdk.py). A LangGraph pipeline (app/graph.py) runs: `ground` → `supervisor` → `[architect | terraform | cost | security specialists in parallel]` → `synthesize`. The supervisor LLM routes the question to relevant specialists; each specialist (app/agents/*.py) is a narrow LLM prompt grounded only in the diagram/Terraform/validation/billing data handed to it; synthesize.py merges their answers into one reply. app/runner.py runs this as a fire-and-forget background task per run, posting progress/results back to Express.

Node / Express additions — tool gateway & system of record

• agentIdentity.js — mints a short-lived (5 min), scoped machine JWT per run (workspace/run/user/scopes), distinct from user session tokens.

• agentBridge.js (startAgentRun) — resolves the diagram, mints the token, and POSTs to the Python service's /runs/start, only waiting for acceptance, not completion.

• internalAgentToolsController.js + routes/internalAgentTools.js — expose /internal/agent-tools/* endpoints, gated by requireAgentScope.js, which verifies the JWT and enforces scopes (e.g. diagram:read, aws:read:billing). Provides a batched grounding endpoint (diagram, generated Terraform, validation, deployment plan, AWS insights) plus runs/:id/steps and runs/:id/complete callbacks, using atomic Mongo updates to avoid races between parallel specialists.

• awsInsightsForAgent.js — reuses the existing cached AWS billing query for the Cost agent.

• AgentRun.js — new Mongo model tracking each run's status/route/steps/findings, polled by the browser.

• agentController.js / agentRoutes.js / index.js — modified to create an AgentRun per message, call startAgentRun instead of the old answerCloudQuestion, and add GET /agent/runs/:id/status for polling; if the Python service is unreachable the run fails gracefully with an inline error.

Frontend

agentApi.ts adds types and getAgentRunStatus; AgentPage.tsx now polls run status every 1.5s (capped at 90s) instead of a single blocking request, rendering the new AgentRunTimeline.tsx component, which shows a live routing → per-specialist → synthesizing progress list with per-agent colors and finding counts. dashboardData.ts adds new suggested prompts (“Review this architecture”, “What's my riskiest resource?”).

End-to-End Flow

• 1. Browser sends a chat message.

• 2. Express creates an AgentRun document and mints a scoped JWT.

• 3. Express calls FastAPI's /runs/start to schedule a LangGraph run.

• 4. FastAPI calls back into Express's /internal/agent-tools/grounding (using that same JWT) to fetch diagram, Terraform, validation, and billing data.

• 5. The supervisor routes the question to relevant specialist agents (architect, terraform, cost, security) — LLM calls only, no direct AWS/Terraform access.

• 6. Specialists post step/complete callbacks to Express, which atomically updates the AgentRun and appends the assistant reply to the conversation.

• 7. The browser polls /agent/runs/:id/status and renders the AgentRunTimeline until the run reaches completed or failed.

Generated summary based on repository exploration of services/agents/, IAAS backend agent integration files, and dashboard frontend changes.